

IT-Lehre Handlungskompetenzen App

Konzept für die produktive Umsetzung

Version 3.0 – inkl. Berechtigungs-, OIDC- und Ausbildungsplatzkonzept

Corporate IT FHNW | Version 3.0 | Mai 2026

Iwo Kuhn – Strategy and Business Alignment

Windisch, 27. Mai 2026

Inhaltsverzeichnis

Inhaltsverzeichnis	2
1 Einleitung	4
2 Funktionaler Umfang	4
2.1 Personenverwaltung (Lernende)	4
2.2 Kompetenzbewertung	4
2.3 Übersicht und Fortschrittsanzeige	4
2.4 Bereichsdetail mit Akkordeon	4
2.5 PDF-Export	4
2.6 Ausbildungsplatzverwaltung und Rotation	5
2.7 KI-Integration (geplant, Phase 4)	5
3 Zielarchitektur	5
3.1 Überblick	5
3.2 Architekturübersicht (schematisch)	5
3.3 Frontend – React	6
3.3.1 Technologie-Stack Frontend	6
3.3.2 Wichtige Komponenten	6
3.4 Backend – .NET Core REST API	6
3.4.1 Technologie-Stack Backend	6
3.4.2 Projektstruktur (Clean Architecture)	7
3.4.3 Authentifizierung und Autorisierung	7
3.5 Datenbank – Microsoft SQL Server	7
4 Berechtigungs- und Rollenkonzept	7
4.1 Rollen	7
4.2 Rollen in Microsoft Entra ID definieren	8
4.3 Lernenden-Verknüpfung (ExternalId)	8
4.4 Datenisolation im Backend	8
4.5 OIDC-Konfiguration	9
4.5.1 Backend (appsettings.json)	9
4.5.2 Frontend (authConfig.ts)	9
5 Ausbildungsplatzverwaltung und Rotation	9
5.1 Übersicht	9
5.2 Ausbildungsplatz-Liste	9
5.3 HK-Bereich-Mapping (Primär / Sekundär)	10
5.4 Rotationsverwaltung	11
5.5 Auswirkung auf die HK-Ansicht	11
6 Datenmodell	11

7 API-Design.....	12
8 Migrations- und Umsetzungsplan	14
8.1 Phase 1 – Fundament inkl. Auth (ca. 4–5 Wochen)	14
8.2 Phase 2 – Kernfunktionen (ca. 5–6 Wochen).....	14
8.3 Phase 3 – Ausbildungsplatzverwaltung (ca. 4 Wochen).....	14
8.4 Phase 4 – KI-Integration & Betrieb (ca. 3 Wochen).....	15
9 Technologieentscheide	15
10 Offene Punkte und nächste Schritte	16
10.1 Offene Punkte.....	16
10.2 Nächste Schritte	16
11 Anhang	17
11.1 Rollen- und Rechteübersicht (kompakt).....	17
11.2 Frontend-Technologie-Stack – Erklärung.....	17
11.2.1 React 18+ mit TypeScript	17
11.2.2 MSAL-React (@azure/msal-browser + @azure/msal-react).....	17
11.2.3 TanStack Query (React Query).....	17
11.2.4 Gesamtbild.....	18
11.3 Review des Konzepts	18
11.3.1 Grundsätzliche Einschätzung.....	18
11.3.2 Kritische Hinterfragung des Tech-Stacks (Overengineering?)	18
11.3.3 Kritische Punkte & Risiken, die du adressieren solltest	18
11.3.4 Zusammenfassung.....	19

1 Einleitung

Die IT-Lehre Handlungskompetenzen App unterstützt Berufsbildende und Lernende in der Informatik-Ausbildung (EFZ) bei der systematischen Erfassung und Verfolgung von Handlungskompetenzen gemäss Bildungsplan 88611. Die App bewertet Leistungsziele nach der Bloom-Taxonomie (K0–K6), visualisiert den Lernfortschritt und ermöglicht den Export strukturierter PDF-Berichte.

Ein lauffähiger Prototyp wurde in zwei Phasen umgesetzt: Phase 1 als HTML-Single-Page-App mit JSON-Datenhaltung auf dem Netzlaufwerk; Phase 2 mit einem Python/FastAPI-Backend und Microsoft SQL Server. Der Prototyp hat die fachlichen Anforderungen validiert und dient als Basis für die vorliegende Konzeption.

Gemäss den architektonischen Vorgaben des Enterprise Architekten basiert die produktive Version auf demselben Stack wie die LPM-App: .NET Core REST API (Backend), React (Frontend) und Microsoft SQL Server. Version 3.0 dieses Dokuments ergänzt das Konzept um ein vollständiges Berechtigungskonzept mit OIDC-Authentifizierung sowie die Ausbildungsplatzverwaltung: Lernende rotieren während ihrer Ausbildung zwischen verschiedenen Ausbildungsplätzen; die App bildet diese Rotation ab und ermöglicht eine ausbildungsplatzspezifische Sicht auf die Handlungskompetenzen.

2 Funktionaler Umfang

2.1 Personenverwaltung (Lernende)

Berufsbildende verwalten Lernende mit Name, Fachrichtung, Lehrbeginn und E-Mail. Unterstützte Fachrichtungen:

- Plattformentwicklung (Bereiche A–F, 4 Lehrjahre)
- Applikationsentwicklung (Bereiche A–C und G–H, 4 Lehrjahre)
- ICT-Fachmann/-frau EFZ (alle Bereiche, 3 Lehrjahre)

Beim Anlegen eines Lernenden wird dessen Azure-AD-ObjectId (OIDC-Sub-Claim) hinterlegt, damit der Lernende beim Login automatisch seinem Datensatz zugeordnet wird.

2.2 Kompetenzbewertung

Jedes Leistungsziel wird nach der Bloom-Taxonomie auf einer Skala von K0 (nicht bearbeitet) bis K6 (Werten) bewertet. Ein Kommentar und das Datum der letzten Bearbeitung können hinterlegt werden. Lernende können ihre eigenen Bewertungen selbst eintragen; der Berufsbildende sieht und prüft sie.

2.3 Übersicht und Fortschrittsanzeige

- Lehrzeit-Fortschrittsbalken (Lehrjahre: absolviert, aktuell, zukünftig)
- Bereichskacheln mit Fortschrittsbalken in Bereichsfarbe
- Donut-Diagramm: Anteil erfüllter Leistungsziele je Bereich und Gesamtfortschritt

2.4 Bereichsdetail mit Akkordeon

Handlungskompetenzen eines Bereichs als aufklappbare Akkordeon-Blöcke mit Bloom-Bewertungsknopf, Kommentarfeld und Speicherdatum. Design gemäss FHNW Corporate Identity.

2.5 PDF-Export

Übersichtskarte (Name, Fachrichtung, Gesamtfortschritt, Donut-Chart), Bloom-Farblegende und Leistungsstand je Leistungsziel mit Rot-Gelb-Grün-Kodierung. Optional: Filter auf aktuellen Ausbildungsplatz.

2.6 Ausbildungsplatzverwaltung und Rotation

Lernende rotieren während ihrer Ausbildung zwischen Ausbildungsplätzen: Im 1. Lehrjahr sind Applikations- und Plattformentwickler im Basislehrjahr (Baden oder Muttentz); ab dem 2. bis 4. Lehrjahr wechseln sie zu den Ausbildungsplätzen der FHNW Corporate IT. Die App bildet diese Rotation ab:

- **Rotationshistorie:** Jeder Ausbildungsplatz-Aufenthalt wird mit Von/Bis-Datum gespeichert. Der aktuelle Ausbildungsplatz ist derjenige ohne Bis-Datum.
- **HK-Bereich-Verknüpfung:** Jedem Ausbildungsplatz sind HK-Bereiche (A–H) zugeordnet (Primär / Sekundär), basierend auf der FHNW-Zuordnungsmatrix.
- **Ausbildungsplatzspezifische Sicht:** Berufsbildner und Lernende können die HK-Ansicht auf die relevanten Bereiche des aktuellen Ausbildungsplatzes einschränken.
- **Filter für Berufsbildner:** Berufsbildner können die Lernenden-Liste nach aktivem Ausbildungsplatz filtern und direkt in die AP-gefilterte HK-Ansicht springen.
- **AP-Verwaltung:** Berufsbildner können die Zuordnung von HK-Bereichen zu Ausbildungsplätzen im Admin-Bereich anpassen.

2.7 KI-Integration (geplant, Phase 4)

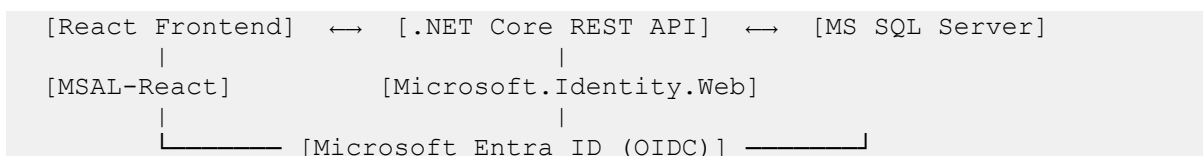
Im Rahmen des CAS AI Software Engineering: automatische Kompetenzstufen-Empfehlungen, Lernfortschritt-Analysen und natürlichsprachige Auswertungen.

3 Zielarchitektur

3.1 Überblick

Die Zielarchitektur folgt einer dreistufigen Architektur (Three-Tier) mit klarer Trennung zwischen Präsentationsschicht, Geschäftslogik und Datenhaltung. Die Kommunikation zwischen Frontend und Backend erfolgt ausschliesslich über eine REST-API mit JSON. Authentifizierung und Autorisierung werden über OIDC mit Microsoft Entra ID als Identity Provider abgewickelt.

3.2 Architekturübersicht (schematisch)



Schicht	Technologie	Aufgabe
Frontend	React (TypeScript)	UI, Routing, State Management, API-Calls
Auth Frontend	MSAL-React	OIDC Login/Logout, Token-Handling im Browser
Backend	.NET Core 8 Web API (C#)	Business Logic, Validierung, REST-Endpunkte, Rollenprüfung

Auth Backend	Microsoft.Identity.Web	JWT-Validierung, Rollenanspruch, Policy-based Auth
Datenbank	Microsoft SQL Server	Persistenz aller Kompetenzdaten, AP-Rotation, Seed-Daten
Identity Provider	Microsoft Entra ID	OIDC-Token-Ausstellung, App-Rollen, Gruppenzuweisung

3.3 Frontend – React

Das Frontend wird als Single-Page Application (SPA) mit React und TypeScript realisiert. Das FHNW Corporate Identity Design und das Komponentenmodell aus Phase 2 werden 1:1 übernommen. Die Authentifizierung übernimmt MSAL-React; nach dem Login stellt die Bibliothek den Access Token für alle API-Requests bereit.

3.3.1 Technologie-Stack Frontend

- React 18+ mit TypeScript
- React Router (react-router-dom) für clientseitiges Routing
- TanStack Query (React Query) für Server-State und Caching
- Tailwind CSS für das Styling (analog LPM-App)
- Axios oder Fetch API für HTTP-Requests (mit Bearer-Token-Interceptor)
- Vite als Build-Tool
- @azure/msal-browser + @azure/msal-react für OIDC / Microsoft Entra ID

3.3.2 Wichtige Komponenten

- **AuthGuard** – Wrapper; leitet nicht angemeldete Nutzer zum Login um
- **RoleGuard** – Versteckt Navigation und Routen, die der aktuellen Rolle nicht gehören
- **App.jsx / AppContext** – Router-Setup, globaler State inkl. currentUser, Rolle und aktiver Ausbildungsplatz
- **Header** – FHNW-Header: Lernenden-Auswahl (nur BB), AP-Badge mit aktuellem Ausbildungsplatz
- **APFilter** – Filter-Chips für Berufsbildner (Lernendenliste nach AP filtern)
- **APToggle** – Toggle «Nur AP-relevante Bereiche» in AreaView und Overview
- **RotationModal** – Modal zum Setzen eines neuen Ausbildungsplatzes (Von-Datum, AP-Auswahl)
- **Overview, AreaView, GoalRow** – Fachliche Komponenten, erweitert um AP-Filterlogik
- **UserModal** – Neu: E-Mail-Feld für OIDC-Linking, Ausbildungsplatz-Zuweisung

3.4 Backend – .NET Core REST API

Das Backend wird als ASP.NET Core 8 Web API in C# implementiert. Es validiert JWT-Bearer-Tokens von Entra ID und prüft Rollenansprüche für jeden Endpunkt. Datenzugriffe werden rollenabhängig gefiltert. Die Ausbildungsplatz-Logik (Rotation, HK-Bereich-Mapping) ist vollständig im Backend gekapselt.

3.4.1 Technologie-Stack Backend

- ASP.NET Core 8 Web API (C#)
- Entity Framework Core 8 als ORM mit MS SQL-Provider

- Microsoft.Identity.Web für OIDC / JWT-Validierung und Policy-based Authorization
- AutoMapper für DTO-Mapping
- FluentValidation für Eingabevalidierung
- Swagger / OpenAPI für API-Dokumentation (inkl. OAuth2 Authorize-Button)
- Serilog für strukturiertes Logging
- xUnit für Unit- und Integrationstests

3.4.2 Projektstruktur (Clean Architecture)

- **HK.API** – Controller, Middleware, Swagger-Konfiguration, Auth-Policies
- **HK.Application** – Use Cases, DTOs, Service-Interfaces
- **HK.Domain** – Entitäten, Domain-Logik, Repository-Interfaces
- **HK.Infrastructure** – EF Core DbContext, Repositories, Seed-Daten (Ausbildungsplätze + HK-Mapping)
- **HK.Tests** – Unit- und Integrationstests inkl. Auth- und AP-Tests

3.4.3 Authentifizierung und Autorisierung

Die API wird vollständig mit Microsoft Entra ID abgesichert. JWT Bearer Tokens werden für alle Endpunkte verlangt. Die Konfiguration erfolgt über Microsoft.Identity.Web:

```
// Program.cs
builder.Services.AddMicrosoftIdentityWebApiAuthentication(builder.Configuration);
builder.Services.AddAuthorization(options => {
    options.AddPolicy("Berufsbildner",
        p => p.RequireRole("Berufsbildner"));
    options.AddPolicy("LernenderOderBB",
        p => p.RequireRole("Lernender", "Berufsbildner"));
});
```

3.5 Datenbank – Microsoft SQL Server

Microsoft SQL Server (Express) mit Entity Framework Core. Der bestehende Datenbankschema aus Phase 2 wird übernommen und um die Ausbildungsplatz-Tabellen erweitert. Alle Ausbildungsplätze und ihre HK-Bereich-Zuordnungen werden als EF Core Seed-Daten eingespielt und können im Admin-Bereich angepasst werden.

4 Berechtigungs- und Rollenkonzept

4.1 Rollen

Es werden zwei Rollen definiert. Sie werden als App Roles in der Entra-ID-App-Registrierung angelegt und Nutzern bzw. Gruppen zugewiesen:

Rolle	Wert im Token (roles-Claim)	Rechte
Berufsbildner	Berufsbildner	• Vollzugriff auf alle Lernenden • Lernende anlegen, bearbeiten, löschen • Rotation (Ausbildungsplatzwechsel) verwalten • Alle Kompetenzbewertungen einsehen • AP-Bereich-Zuordnung verwalten

Lernender	Lernender	• Ausschliesslich eigene Daten sichtbar • Eigene Kompetenzbewertungen eintragen • Eigenen AP-Badge und Rotationshistorie sehen • HK-Ansicht auf eigenen AP filterbar
------------------	-----------	---

4.2 Rollen in Microsoft Entra ID definieren

1. Azure Portal → App-Registrierungen → HK-App → App-Rollen
2. Neue Rolle «Berufsbildner» erstellen: Wert = Berufsbildner, Erlaubt für = Benutzer/Gruppen
3. Neue Rolle «Lernender» erstellen: Wert = Lernender, Erlaubt für = Benutzer/Gruppen
4. Unter Unternehmensanwendungen → Benutzer und Gruppen: Zuweisung per Person oder AAD-Gruppe

Nach dem Login ist der roles-Claim automatisch im JWT enthalten und wird sowohl im Frontend (Navigation) als auch im Backend (Policies) ausgewertet.

4.3 Lernenden-Verknüpfung (ExternalId)

Damit ein eingeloggter Lernender automatisch seinem Datensatz zugeordnet wird, speichert die App die AAD-ObjectId (oid-Claim) in der Spalte Users.ExternalId. Der Ablauf beim ersten Login:

Schritt	Beschreibung
1	Berufsbildner legt Lernenden an (Name, Fachrichtung, Lehrbeginn, E-Mail) und weist den ersten Ausbildungsplatz zu.
2	Beim ersten Login liefert Entra ID den oid-Claim im JWT.
3	Backend sucht anhand der oid in Users.ExternalId. Kein Treffer → Suche über E-Mail (upn-Claim). Bei Treffer: ExternalId automatisch gesetzt.
4	Ab sofort filtert das Backend alle Abfragen auf UserId = verknüpfter Datensatz.
5	Falls kein Datensatz gefunden: Hinweisseite «Ihr Berufsbildner hat Sie noch nicht erfasst.»

4.4 Datenisolation im Backend

Endpunkt	Verhalten je Rolle
GET /api/v1/users	Berufsbildner: alle (optional Filter ?ausbildungsplatz=). Lernender: nur eigener Datensatz.
GET /api/v1/users/{id}	Berufsbildner: immer. Lernender: nur eigene UserId, sonst 403.
GET /api/v1/users/{id}/goals	Gleiche Logik wie oben.
PUT /api/v1/users/{id}/goals/{goalId}	Lernender: nur eigene. Berufsbildner: alle.
POST/PUT/DELETE /api/v1/users	Nur Berufsbildner.

POST /api/v1/users/{id}/rotations	Nur Berufsbildner (AP-Wechsel auslösen).
GET /api/v1/users/{id}/rotations	Beide – Lernender nur eigene History.
GET /api/v1/me	Beide – eigenen Datensatz anhand oid-Claim abrufen.

4.5 OIDC-Konfiguration

4.5.1 Backend (appsettings.json)

```
{
  "AzureAd": {
    "Instance": "https://login.microsoftonline.com/",
    "TenantId": "<FHNW-Tenant-ID>",
    "ClientId": "<HK-App-Client-ID>",
    "Audience": "api://<HK-App-Client-ID>"
  }
}
```

4.5.2 Frontend (authConfig.ts)

```
export const msalConfig = {
  auth: {
    clientId: "<HK-App-Client-ID>",
    authority: "https://login.microsoftonline.com/<FHNW-Tenant-ID>",
    redirectUri: "http://localhost:5173"
  }
};
export const loginRequest = {
  scopes: ["api://<HK-App-Client-ID>/access_as_user"]
};
```

5 Ausbildungsplatzverwaltung und Rotation

5.1 Übersicht

Lernende der Informatik-Ausbildung durchlaufen während ihrer Ausbildungszeit mehrere Ausbildungsplätze. Im 1. Lehrjahr befinden sich Applikations- und Plattformentwickler im Basislehrjahr (Baden oder Muttentz). Ab dem 2. Lehrjahr rotieren sie zu den Ausbildungsplätzen der FHNW Corporate IT. Jeder Ausbildungsplatz vermittelt dabei einen spezifischen Teil der Handlungskompetenzen gemäss Bildungsplan 88611.

Die App bildet diese Rotation ab: Berufsbildner setzen den jeweils aktuellen Ausbildungsplatz eines Lernenden. Sowohl die Lernenden-Ansicht als auch die Berufsbildner-Sicht können auf die Bereiche des aktuellen Ausbildungsplatzes eingeschränkt werden. Die Verknüpfung zwischen Ausbildungsplätzen und HK-Bereichen ist etwas grösser gefasst als die granulare Zuordnungsmatrix im XLSX — sie operiert auf Bereich-Ebene (A–H) statt auf einzelnen Leistungszielen.

5.2 Ausbildungsplatz-Liste

Kürzel	Bezeichnung	Hinweis
--------	-------------	---------

BLB	Basislehrjahr Baden	1. Lehrjahr; alle HK-Bereiche als Grundlagenjahr
BLM	Basislehrjahr Muttentz	1. Lehrjahr; alle HK-Bereiche als Grundlagenjahr
SDM	IT-Servicedesk Muttentz	Ab 2. LJ; Bereiche a, b, c, d (Support & Betrieb)
SDO	IT-Servicedesk Olten	Ab 2. LJ; Bereiche a, b, c, d
SDW	IT-Servicedesk CBW	Ab 2. LJ; Bereiche a, b, c, d
WE	Workplace Engineering	Ab 2. LJ; Bereiche b, c, d, h
COL	Collaboration	Ab 2. LJ; Bereiche a, b, c, d, f, g
EP	Enterprise Platforms	Ab 2. LJ; Bereiche a, b, c, d, f, h
WEB	Web	Ab 2. LJ; Bereiche a, b, c, d, g, h
SWE	Software Engineering	Ab 2. LJ; Bereiche a, c, g, h
SA	Fachbereich Schuladmin	Ab 2. LJ; Bereiche a, b, c, d, f, g, h
NetDC	Network & Datacenter	Ab 2. LJ; Bereiche b, c, d, e, f
CIS	Cloud Infrastruktur Services	Ab 2. LJ; Bereiche a, c, d, f
PnD	Protection and Design	Ab 2. LJ; Bereiche a, b, c
SBA	Strategy & Business Alignment	Ab 2. LJ; Bereiche a, c (pragmatisch ergänzt)
PnC	People & Culture	Ab 2. LJ; Bereiche b, d (pragmatisch ergänzt)

5.3 HK-Bereich-Mapping (Primär / Sekundär)

Die Zuordnung der HK-Bereiche zu den Ausbildungsplätzen basiert auf der FHNW-Zuordnungsmatrix (XLSX) und ist auf Bereich-Ebene A–H vereinfacht. Primäre Bereiche sind der Hauptfokus des Ausbildungsplatzes; sekundäre Bereiche werden ergänzend behandelt. Ausbildungsplätze ohne Eintrag im XLSX (SBA, PnC) wurden pragmatisch ergänzt.

Kürzel	Ausbildungsplatz	Primäre Bereiche	Sekundäre Bereiche
BLB / BLM	Basislehrjahr Baden / Muttentz	a, b, c, d, e, f, g, h	(alle primär — Grundlagenjahr)
SDM/SDO/SDW	IT-Servicedesk (alle Standorte)	a, b, c, d	—
WE	Workplace Engineering	b, c, d, h	—
COL	Collaboration	a, b, c, d, f, g	e
EP	Enterprise Platforms	a, b, d, f, h	c, e, g
WEB	Web	a, b, d, g, h	c
SWE	Software Engineering	a, g, h	b, c, d
SA	Fachbereich Schuladmin	a, b, c, f, h	d, e, g
NetDC	Network & Datacenter	b, c, f	d, e
CIS	Cloud Infrastruktur Services	a, f	c, d

PnD	Protection and Design	a, b, c	–
SBA	Strategy & Business Alignment	a, c	–
PnC	People & Culture	b, d	–

Das Mapping ist als EF Core Seed-Daten hinterlegt und kann im Admin-Bereich der App jederzeit durch Berufsbildner angepasst werden (Checkbox-Liste A–H, Primär/Sekundär-Auswahl je Bereich).

5.4 Rotationsverwaltung

Der Berufsbildner setzt den Ausbildungsplatz eines Lernenden über einen Wechsel-Dialog. Dabei wird die aktive Rotation automatisch abgeschlossen und eine neue eröffnet:

```
// Pseudocode - AP-Wechsel
POST /api/v1/users/{id}/rotations
{
  "ausbildungsplatzCode": "SWE",
  "von": "2025-08-01"
}
// Backend:
// 1. Aktive Rotation (Bis = null) → Bis = von - 1 Tag
// 2. Neue Rotation anlegen (Von = von, Bis = null)
```

Die Rotationshistorie (alle vergangenen und aktive Rotation) ist für Berufsbildner vollständig einsehbar. Lernende sehen ihre eigene History.

5.5 Auswirkung auf die HK-Ansicht

Der AP-Filter ist ein optionaler Toggle — er verändert keine Daten, sondern steuert nur die Sichtbarkeit:

Sicht / Toggle	Verhalten
Vollsicht (Standard)	Alle Bereiche A–H sichtbar, wie bisher.
«Nur aktiver AP»	Nur Bereiche angezeigt, die dem aktuellen AP zugeordnet sind (primär und sekundär). Nicht zugeordnete Bereiche ausgegraut oder versteckt.
Berufsbildner: AP-Filter	Filter-Chip in der Lernenden-Liste. Zeigt nur Lernende mit aktiver Rotation an diesem AP. Klick auf einen Lernenden öffnet direkt AP-gefilterte HK-Ansicht.
PDF-Export	Optional: PDF auf aktuellen AP einschränken (nur relevante Bereiche im Bericht).

6 Datenmodell

Das Datenmodell basiert auf dem in Phase 2 bewährten Schema und wird um die Ausbildungsplatz-Entitäten erweitert. Users.ExternalId verknüpft Lernende mit ihrem AAD-Konto (OIDC).

Entität	Beschreibung	Wichtige Felder
---------	--------------	-----------------

User	Lernende Person	Id (GUID), Name, Specialty, StartDate, CreatedAt, ExternalId (AAD ObjectId), Email
GoalEntry	Kompetenzbewertung je Lernziel	Id, UserId (FK, CASCADE), GoalId (z.B. a1.1), Level (0–6), Comment, UpdatedAt. UNIQUE (UserId, GoalId).
Competency	Leistungsziel aus Bildungsplan 88611	Id, GoalId, AreaCode, Title, MaxLevel, Specialty
Area	Handlungskompetenzbereich	Code (A–H), Name, Color, Specialties (m:n)
Ausbildungsplatz	Ausbildungsplatz der FHNW CIT	Id (GUID), Code (BLB, SDM ...), Bezeichnung, AbLehrjahr (1 oder 2)
AusbildungsplatzHKBereich	HK-Bereich-Zuordnung pro AP	AusbildungsplatzCode (FK), AreaCode (FK), IsPrimary (bool). PK: (APCode, AreaCode). EF Core Seed.
UserRotation	Rotationshistorie eines Lernenden	Id (GUID), UserId (FK), AusbildungsplatzCode (FK), Von (DATE), Bis (DATE?) – null = aktive Rotation.

EF Core Migration (Auszug) – neue Tabellen:

```
// Ausbildungsplatz
migrationBuilder.CreateTable("Ausbildungsplaetze", t => new {
    Id = t.Column<Guid>(),
    Code = t.Column<string>(maxLength: 10),
    Bezeichnung = t.Column<string>(maxLength: 100),
    AbLehrjahr = t.Column<int>() });

// AusbildungsplatzHKBereich (Seed-Daten aus XLSX-Mapping)
migrationBuilder.CreateTable("AusbildungsplatzHKBereiche", t => new {
    AusbildungsplatzCode = t.Column<string>(maxLength: 10),
    AreaCode = t.Column<string>(maxLength: 1),
    IsPrimary = t.Column<bool>() });

// UserRotation
migrationBuilder.CreateTable("UserRotations", t => new {
    Id = t.Column<Guid>(),
    UserId = t.Column<Guid>(),
    AusbildungsplatzCode = t.Column<string>(maxLength: 10),
    Von = t.Column<DateOnly>(),
    Bis = t.Column<DateOnly>(nullable: true) });
```

7 API-Design

Alle Endpunkte verlangen einen gültigen JWT Bearer Token. Neue Endpunkte für Ausbildungsplätze und Rotationsverwaltung sind als eigene Ressourcen strukturiert.

Methode	Pfad	Rolle	Beschreibung
---------	------	-------	--------------

GET	/api/v1/users	Beide	BB: alle (opt. ?ausbildungsplatz=SDM). Lernender: eigener.
POST	/api/v1/users	BB	Neuen Lernenden anlegen (inkl. Email, erster AP).
GET	/api/v1/users/{id}	Beide	Detail – Lernender nur eigener, BB alle.
PUT	/api/v1/users/{id}	BB	Lernenden bearbeiten.
DELETE	/api/v1/users/{id}	BB	Lernenden löschen (inkl. Goals und Rotationen).
GET	/api/v1/users/{id}/goals	Beide	Bewertungen – eigene / alle.
PUT	/api/v1/users/{id}/goals/{goalId}	Beide	Bewertung speichern – nur eigene für Lernende.
GET	/api/v1/users/{id}/progress	Beide	Fortschritt je Bereich (opt. ?ausbildungsplatz=).
GET	/api/v1/users/{id}/report	Beide	PDF-Reportdaten (opt. ?ausbildungsplatz=).
GET	/api/v1/users/{id}/rotations	Beide	Rotationshistorie – Lernender nur eigene.
POST	/api/v1/users/{id}/rotations	BB	Neuen AP setzen; aktive Rotation wird automatisch geschlossen.
GET	/api/v1/me	Beide	Eigenen Datensatz anhand oid-Claim abrufen / OIDC-Linking.
GET	/api/v1/ausbildungsplaetze	Beide	Alle 16 Ausbildungsplätze abrufen.
GET	/api/v1/ausbildungsplaetze/{code}/bereiche	Beide	HK-Bereiche (A–H) für einen AP (primär/sekundär).
PUT	/api/v1/ausbildungsplaetze/{code}/bereiche	BB	HK-Bereich-Zuordnung für einen AP anpassen.
GET	/api/v1/competencies/{specialty}	Beide	Leistungsziele für Fachrichtung.
GET	/api/v1/areas	Beide	Alle HK-Bereiche.

8 Migrations- und Umsetzungsplan

Die Umsetzung erfolgt in vier Phasen. Der bestehende Prototyp (Phase 2) bleibt parallel in Betrieb, bis Phase 2 der produktiven Version abgeschlossen ist.

8.1 Phase 1 – Fundament inkl. Auth (ca. 4–5 Wochen)

Ziel: Infrastruktur, Grundgerüst und Authentifizierung bereitstellen.

- Repository-Struktur und Branching-Strategie aufsetzen
- ASP.NET Core 8 Web API Projekt anlegen (Clean Architecture HK.*)
- Entity Framework Core mit MS SQL einrichten, Migrations ausführen
- ExternalId-Spalte und Index in EF Core Migration anlegen
- Datenmigration: Bestehende MSSQL-Daten aus Phase 2 übernehmen
- Kompetenz-JSONs (Bildungsplan 88611) als Seed-Daten importieren
- React-Projekt aufsetzen (Vite, TypeScript, Tailwind CSS); FHNW CI Design 1:1 übernehmen
- Entra-ID App-Registrierung einrichten (Rollen Berufsbildner / Lernender)
- MSAL-React im Frontend und Microsoft.Identity.Web im Backend konfigurieren
- Grundlegender Auth-Fluss End-to-End testen

8.2 Phase 2 – Kernfunktionen (ca. 5–6 Wochen)

Ziel: Alle Prototyp-Funktionen produktionsreif umsetzen.

- REST-API Endpunkte für alle Entitäten implementieren (inkl. Rollenpolicies)
- Datenisolation: alle Service-Abfragen auf ExternalId / UserId filtern
- /api/v1/me Endpunkt für OIDC-Linking implementieren
- React-Komponenten: Overview, AreaView, GoalRow, UserModal (inkl. E-Mail-Feld)
- RoleGuard und AuthGuard im Frontend implementieren
- Berufsbildner-Ansicht: Lernenden-Dropdown im Header; Lernenden-Ansicht: direkt eigene Daten
- PDF-Export serverseitig (.NET)
- Integrations- und Unit-Tests inkl. Rollenszenarien (Ziel: $\geq 80\%$ Coverage)
- User-Acceptance-Test mit Berufsbildenden und Lernenden

8.3 Phase 3 – Ausbildungsplatzverwaltung (ca. 4 Wochen)

Ziel: Ausbildungsplätze, Rotation und ausbildungsplatzspezifische HK-Sicht implementieren.

- EF Core Migration für Ausbildungsplatz, AusbildungsplatzHKBereich und UserRotation
- Seed-Daten: alle 16 Ausbildungsplätze und HK-Bereich-Mapping aus XLSX importieren
- REST-API Endpunkte für Ausbildungsplätze und Rotationsverwaltung implementieren
- Rotationslogik im Backend: aktive Rotation schliessen, neue eröffnen
- Frontend: RotationModal (AP-Wechsel), AP-Badge im Header, Rotationshistorie-Ansicht
- APFilter-Chips in der Berufsbildner-Lernendenliste
- APToggle in AreaView und Overview («Nur AP-relevante Bereiche»)
- Admin-Bereich: HK-Bereich-Zuordnung pro AP anpassen (Checkbox-Liste A–H)
- PDF-Export: optionaler AP-Filter im Bericht

- End-to-End-Tests: Rotationswechsel, AP-Filter, Sichtbarkeitslogik

8.4 Phase 4 – KI-Integration & Betrieb (ca. 3 Wochen)

Ziel: KI-Erweiterungen im Rahmen des CAS AI-SE und Übergabe in den Betrieb.

- KI-gestützte Kompetenzstufen-Empfehlung (OpenAI API oder lokales Modell)
- Natürlichsprachige Lernfortschritt-Auswertung
- Performance-Optimierung (Caching, SQL-Indexes auf Rotation und GoalEntry)
- Deployment auf Zielumgebung (lokal oder FHNW-Server)
- Betriebsdokumentation und Übergabe

9 Technologieentscheide

Entscheid	Gewählte Option	Begründung
Backend Framework	.NET Core 8 Web API	Identisch mit LPM-App → kein zusätzlicher Lernaufwand; Vorgabe Enterprise Architektur
Frontend Framework	React 18 + TypeScript	Bereits im Prototyp umgesetzt; identisch mit LPM-App
Datenbank	Microsoft SQL Server	Bereits vorhanden (SQLEXPRESS); Vorgabe Enterprise Architektur
ORM	Entity Framework Core 8	Code-First Migrations; AP- und Rotation-Tabellen einfach erweiterbar
Identity Provider	Microsoft Entra ID	FHNW nutzt M365 → kein zusätzliches Identity-System; SSO mit FHNW-Account
Auth-Protokoll	OIDC (OpenID Connect)	Industriestandard; von Entra ID nativ unterstützt; kein eigenes Session-Management
Frontend Auth	MSAL-React	Offizielle Microsoft-Bibliothek; unterstützt SPA-Auth-Flow, Token-Caching, Silent Refresh
Backend Auth	Microsoft.Identity.Web	Ein NuGet-Paket; kapselt JWT-Validierung und Policy-Auth vollständig
Rollen-Mechanismus	Entra ID App Roles	Rollenzuweisung in AAD ohne eigene Rollentabelle; roles-Claim direkt im JWT
AP-HK-Mapping	EF Core Seed + Admin-UI	Vorbelegt aus XLSX-Matrix, jederzeit durch Berufsbildner anpassbar ohne Code-Änderung
Rotationshistorie	UserRotation Tabelle	Von/Bis-Datumspaar; aktive Rotation = Bis IS NULL; vollständige History erhalten
AP-Granularität	Bereich-Ebene (A–H)	Pragmatisch und wartbar; «etwas grösser gefasst» als Einzelziel-Matrix im XLSX
Styling	Tailwind CSS	Bereits im Prototyp; konsistentes Design mit LPM-App
State Management	TanStack Query	Leichtgewichtig; ideal für Server-State
Build-Tool	Vite	Schneller Dev-Server; HMR; identisch mit LPM-App

PDF-Export	.NET (Backend)	Serverseitig; robuster als jsPDF; inkl. optionalem AP-Filter
------------	----------------	--

10 Offene Punkte und nächste Schritte

10.1 Offene Punkte

- **Entra-ID Berechtigungen:** Wer in der FHNW-IT darf die App-Registrierung und Rollenzuweisungen verwalten?
- **Selbstregistrierung Lernender:** Darf ein Lernender sich selbst anlegen, oder ausschliesslich der Berufsbildner?
- **Initiale Rotationszuweisung:** Wird der erste Ausbildungsplatz beim Anlegen des Lernenden gesetzt oder separat?
- **AP-Mapping Pflege:** Wer ist verantwortlich für die Aktualisierung des HK-Bereich-Mappings wenn sich Ausbildungsplätze ändern?
- **Bildungsplan-Aktualisierungen:** Wie werden Änderungen am Bildungsplan 88611 in die Datenbank eingespielt?
- **Deployment-Ziel:** Lokal (localhost), Schulnetzwerk (IIS/Windows Server) oder Azure App Service?
- **KI-Komponente:** Welches Modell / welche API für die Kompetenzstufen-Empfehlung (CAS AI-SE, Phase 4)?

10.2 Nächste Schritte

5. Review dieses Konzepts (v3.0) mit Berufsbildenden
6. Entra-ID App-Registrierung beantragen / einrichten (Tenant-ID, Client-ID)
7. AAD-Gruppen für Berufsbildner und Lernende definieren
8. Entwicklungsumgebung aufsetzen (Visual Studio / VS Code, .NET SDK 8, SQL Server)
9. ASP.NET Core 8 Projekt anlegen und Microsoft.Identity.Web konfigurieren
10. React-Projekt auf TypeScript umstellen und MSAL-React einführen
11. Umsetzungsstart Phase 1 einplanen

11 Anhang

11.1 Rollen- und Rechteübersicht (kompakt)

Aktion	Berufsbildner	Lernender
Login (OIDC)	✓ FHNW-Account	✓ FHNW-Account
Eigene Daten sehen	✓	✓
Alle Lernenden sehen	✓	✗
Lernenden anlegen / bearbeiten / löschen	✓	✗
Ausbildungsplatz setzen / wechseln	✓	✗
Eigene HK-Ansicht auf AP filtern	✓	✓ (eigener AP)
Lernendenliste nach AP filtern	✓	✗
AP-HK-Bereich-Mapping anpassen	✓	✗
Eigene Kompetenzbewertungen eintragen	✓	✓
Bewertungen anderer Lernenden sehen	✓	✗
Eigenen PDF-Bericht exportieren	✓	✓
PDF aller Lernenden exportieren	✓	✗
KI-Empfehlungen (Phase 4)	✓	✓ (eigene)

11.2 Frontend-Technologie-Stack – Erklärung

11.2.1 React 18+ mit TypeScript

React ist eine komponentenbasierte UI-Bibliothek von Meta. Statt ganze Seiten neu zu rendern, aktualisiert React nur die Teile des DOMs, die sich tatsächlich verändert haben. TypeScript ergänzt JavaScript um statische Typen: Fehler werden beim Schreiben des Codes entdeckt, Refactoring wird sicherer. Der Phase-2-Prototyp nutzt bereits React mit Vite.

11.2.2 MSAL-React (@azure/msal-browser + @azure/msal-react)

MSAL ist die offizielle Microsoft-Bibliothek für OIDC-Auth in Single-Page-Applications. Sie übernimmt den gesamten Auth-Code-Flow, Token-Caching und Silent-Refresh automatisch. Hooks wie `useMsal()` und `useIsAuthenticated()` vereinfachen die Integration in React-Komponenten erheblich.

11.2.3 TanStack Query (React Query)

TanStack Query löst das Problem von Server-Daten, die sich ändern und von mehreren Komponenten gleichzeitig benötigt werden. Im Phase-2-Prototyp wurde dieser Teil noch manuell in `AppContext.jsx` gelöst. TanStack Query vereinfacht das und verwaltet auch die Bearer-Token-Weitergabe über Axios-Interceptoren.

11.2.4 Gesamtbild

Dieser Stack ist 2025/26 der De-facto-Standard für moderne React-Projekte. Entscheidend: der Stack ist identisch mit der LPM-App. Wer einen der beiden Apps weiterentwickelt, kann dieselben Kenntnisse direkt auf die andere anwenden. Das minimiert den gesamten Lernaufwand bei zwei parallelen Projekten.

11.3 Review des Konzepts

Dieses Konzept für die „Handlungskompetenzen-App“ habe ich kritisch geprüft. Hier ist meine Einschätzung zu deinem Vorhaben, dem Technologie-Stack und der Verhältnismäßigkeit.

11.3.1 Grundsätzliche Einschätzung

Das Konzept ist **ausgesprochen solide, professionell strukturiert und durchdacht**. Es ist deutlich mehr als ein "kleines Projekt" – es ist eine sauber spezifizierte Enterprise-Anwendung im Miniaturformat.

- **Stärken:** Die Trennung in Phasen, die klare Architekturvorgabe (Clean Architecture) und das durchdachte Rollen- und Berechtigungskonzept zeigen, dass hier jemand mit Erfahrung am Werk ist.
- **Wartbarkeit:** Dass die App denselben Stack wie die "LPM-App" der FHNW verwendet, ist ein strategischer Volltreffer. Es senkt die langfristigen Betriebskosten und den Einarbeitungsaufwand enorm.

11.3.2 Kritische Hinterfragung des Tech-Stacks (Overengineering?)

Du stellst zu Recht die Frage, ob der Stack für maximal 50 Nutzer nicht "mit Kanonen auf Spatzen geschossen" ist.

- **Backend (.NET Core 8 Web API):** Ja, das ist "Enterprise"-Standard. Für 50 Nutzer wäre eine Node.js/Python-Lösung vielleicht "schneller" hingeschrieben. **Aber:** Da die FHNW-Umgebung (.NET/.SQL Server) vorgegeben ist, ist das die einzig sinnvolle Wahl. Es erspart dir den Betrieb einer heterogenen Infrastruktur.
- **Frontend (React + TypeScript + Tailwind):** Absolut gerechtfertigt. Gerade bei 50 Nutzern (Berufsbildner + Lernende) willst du eine App, die sich "nativ" und schnell anfühlt. React ist hier der Goldstandard für Wartbarkeit.
- **Datenbank (SQL Server):** Das ist der einzige Punkt, der "schwer" wirkt. Wenn du bereits eine SQL-Express-Instanz hast, ist das völlig in Ordnung. Wenn du sie für dieses Projekt *neu* aufsetzen müsstest, wäre eine leichtgewichtige DB (z.B. SQLite) für den Betrieb und das Backup einfacher – aber für eine offizielle FHNW-App ist der SQL Server die richtige Wahl für Compliance und Sicherheit.

Fazit zum Stack: Es ist **kein Overengineering**, sondern **Standardisierung**. Die App ist klein, aber die *Anforderungen an Sicherheit, Rollentrennung und Audit-Fähigkeit* (Bildungsplan 88611) sind hoch. Der gewählte Stack erfüllt diese Anforderungen ohne Reibungsverluste.

11.3.3 Kritische Punkte & Risiken, die du adressieren solltest

Obwohl das Konzept gut ist, gibt es Aspekte, die in der Praxis oft unterschätzt werden:

1. **Das "OIDC-Linking" (Schwachstelle):** Das Konzept sieht vor, dass beim ersten Login die ExternalId (OID) verknüpft wird, falls ein Treffer bei der E-Mail vorliegt.
Risiko: Wenn sich die E-Mail-Adressen im System ändern oder Nutzer verschiedene Accounts haben, kann das automatische Linking fehlschlagen oder fehlerhaft sein.
Empfehlung: Implementiere einen manuellen "Account-Link"-Prozess im Admin-Bereich als Fallback.

2. **Daten-Integrität bei Rotation:** Das Konzept der Rotation mit einem "Bis"-Datum (null = aktiv) ist gut. Achte darauf, dass das Backend strikt verhindert, dass ein Nutzer gleichzeitig in zwei aktiven Rotationen sein kann (eindeutiger Index auf UserId + Bis IS NULL).
3. **Deployment (Die größte Unbekannte):** Punkt 10.1 im Dokument zeigt, dass das Deployment noch offen ist. Für 50 Nutzer ist ein Azure App Service (Shared/Basic) zwar bequem, aber unter Umständen teuer. Ein IIS-Server innerhalb des FHNW-Netzes ist technisch stabiler, erfordert aber Administratoren-Rechte. Kläre das frühzeitig, da dies die API-Konfiguration (CORS, OIDC Redirect URIs) massiv beeinflusst.
4. **Wartung der Seed-Daten:** Dass das Mapping (HK-Bereich zu Ausbildungsplatz) im Admin-Bereich anpassbar ist, ist super. Stelle sicher, dass die App beim Start/Update die "Original"-Matrix aus einem Config-File oder dem Quellcode wiederherstellen kann, falls ein Berufsbildner versehentlich die Konfiguration "kaputtspielt".

11.3.4 Zusammenfassung

Das Konzept ist reif für die Umsetzung. Du baust hier kein Spielzeug, sondern ein Produkt, das die FHNW-Informatik-Ausbildung sauber digitalisiert. Der gewählte Stack ist aufgrund der FHNW-Vorgaben nicht nur legitim, sondern sogar der einzige Weg, um langfristig keine "Technical Debt" anzuhäufen.

Dieses Konzept wurde mit Hilfe von AI erstellt. Kaptiel 1 - 9.1 mit Claude und der Review im Kapitel 9.2 mit Google Gemini.